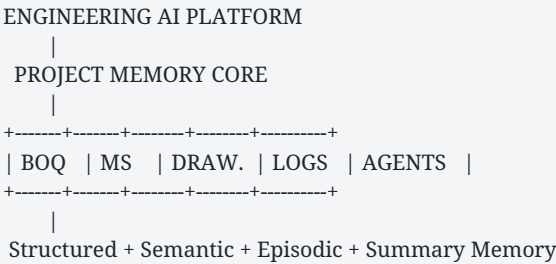


PROJECT MEMORY SYSTEM

Master Plan for an Engineering AI Platform

Persistent, evidence-backed, project-scoped memory for AI agents, RAG, drawings, compliance, BOQ, calculations, and approvals



Contents

1. Executive Summary
2. Vision and Objectives
3. Design Principles
4. Target Architecture
5. Memory Model
6. Data Architecture
7. Evidence and Trust Model
8. Document and RAG Integration
9. Episodic Memory and Project Timeline
10. Project Summary Memory
11. Retrieval and Context Building
12. Conflict, Revision, and Correction Handling
13. Automatic Memory Extraction
14. Memory Ranking and Deduplication
15. User Experience and UI
16. Agent Integration
17. API and Service Design
18. Security, Roles, and Auditability
19. Quality, Testing, and Evaluation
20. Implementation Roadmap M1-M14
21. Deployment Strategy
22. Success Metrics
23. Future Extensions
24. Final Target State

Core rule: The memory system must never mix project-specific facts across projects. Every project memory object is scoped by project_id and backed by evidence wherever possible.

1. Executive Summary

The Project Memory System is a core architectural layer for the Engineering AI Platform. Its purpose is to allow every project to retain a trustworthy operational memory of facts, decisions, approvals, revisions, user corrections, open issues, and historical events without repeatedly sending the entire project history to a large language model.

The design combines deterministic structured data, semantic retrieval, event history, summarized context, and evidence-aware AI. The result is a platform where each project behaves as if an experienced project engineer has continuously followed it from day one.

- **Project isolation.** No project-specific memory is retrieved outside its own project scope.
- **Evidence-first engineering.** Critical facts such as approvals, consultant decisions, authority requirements, and design values are linked to their sources.
- **Token efficiency.** The LLM receives only the minimum relevant context instead of full chat or document history.

- **Revision awareness.** The current state is separated from historical R0/R1/R2 states and superseded information remains auditable.
- **Shared intelligence.** All platform agents consume the same Project Memory Service rather than creating isolated memory stores.
- **Controlled automation.** AI may propose memories, but critical memories are validated before being treated as authoritative.

Target outcome: A user can open any project, ask an engineering question, and receive an answer grounded in the correct project state, the latest revision, approved project evidence, relevant historical decisions, and global engineering knowledge.

2. Vision and Objectives

The memory layer should make the platform persistent, context-aware, explainable, and safe for engineering workflows.

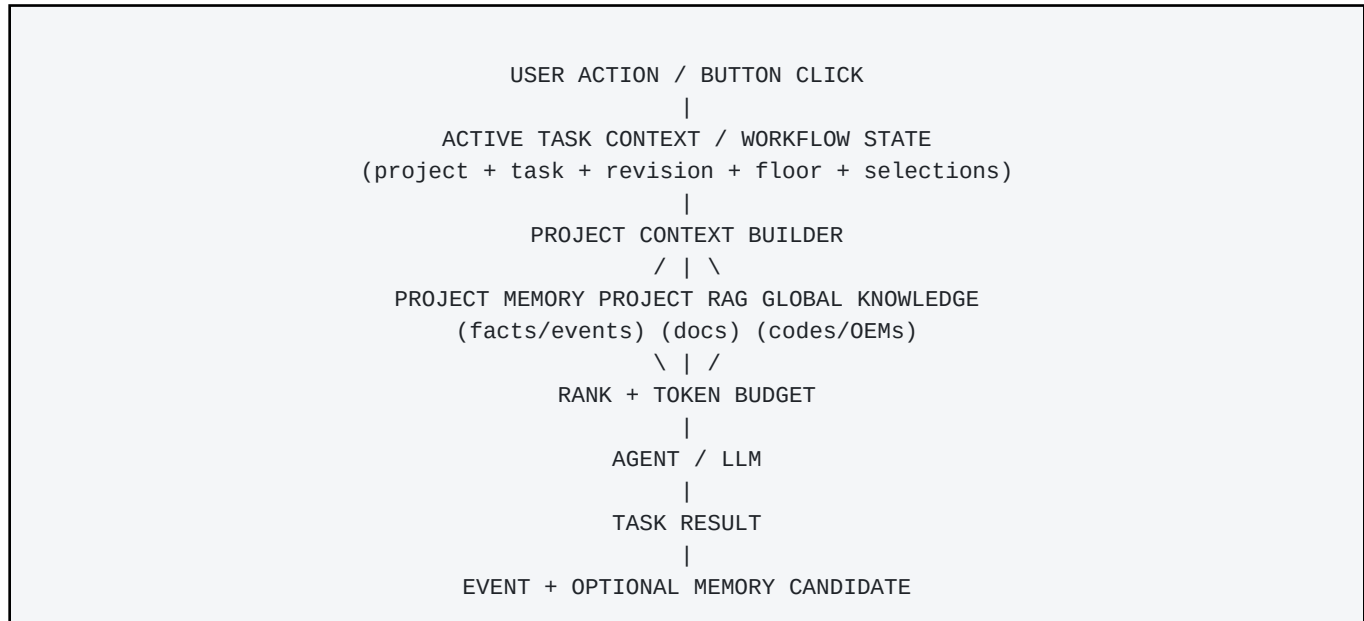
Objective	What it means in practice
Persistent project context	The system remembers confirmed facts, design changes, decisions, and unresolved issues across sessions.
Minimum repeated input	Users do not need to re-explain brands, consultant requirements, loop rules, current revisions, or previous decisions.
Engineering traceability	Every important answer can point back to a structured fact, project event, drawing, submittal, email, specification clause, or user-confirmed decision.
Low token cost	The system retrieves only relevant facts, memories, events, and document chunks for each task.
Cross-agent consistency	Compliance, drawings, BOQ, battery, interface, and shop-drawing agents all use the same memory state.
Controlled learning	User corrections improve future project behavior without silently rewriting company-wide knowledge.

3. Design Principles

1. Project scope first: every project-specific retrieval begins with `project_id`.
2. Deterministic before generative: structured facts and direct database queries are preferred over LLM inference.
3. Evidence before confidence: important memories must retain source, page, revision, author, and timestamp when available.
4. Latest by default, history on demand: current answers use the latest valid state while historical versions remain accessible.
5. Never hard-delete engineering history: supersede, reject, or deactivate records while retaining the audit trail.

6. **Corrections outrank inference:** user-confirmed corrections and approved evidence take priority over model assumptions.
7. **One memory service for all agents:** memory logic belongs to platform infrastructure, not inside individual prompts.
8. **Task context is explicit:** button-driven workflows provide project, task, revision, floor/zone, selected entities, and temporary inputs to the Context Builder.
9. **Token budget is an architectural constraint:** retrieval depth must be managed per task complexity.
10. **Critical false memory rate should approach zero** for approvals, authority requirements, design constraints, and safety-relevant data.

4. Target Architecture



The architecture separates three kinds of state:

Active Task Context. Temporary workflow state for the current action. It is not permanent project memory by default.

Project Knowledge. Project-specific facts, decisions, revisions, evidence, events, approvals, calculations, and corrections.

Global Knowledge. Reusable company rules, codes, standards, manufacturer data, and validated historical patterns.

5. Memory Model

The persistent design still uses four complementary memory layers. Active Task Context sits **before retrieval** and is intentionally ephemeral.

Layer	Purpose	Examples	Primary storage
1. Structured Memory	Authoritative machine-readable project state	Consultant, project type, approved brand, latest revision, system settings	PostgreSQL

Layer	Purpose	Examples	Primary storage
2. Semantic Memory	Searchable project document knowledge	Specification clauses, datasheets, comments, letters, submittals	Vector index + document store
3. Episodic Memory	Chronological record of meaningful project events	Design change, approval, correction, revision update, issue raised	PostgreSQL event log
4. Summary Memory	Compact current project context for LLM use	System summaries, approvals, open issues, recent changes	PostgreSQL / generated cache

Recommended memory types

fact
 decision
 approval
 rejection
 correction
 design_change
 constraint
 lesson
 open_issue
 resolution

6. Data Architecture

A normalized relational model should hold authoritative metadata, while embeddings remain an index rather than the source of truth.

Table	Purpose
projects	Project master record and core metadata.
project_facts	Current structured facts such as consultant, project type, brand, system constraints, and current revision.
project_memories	Human-readable facts, decisions, corrections, approvals, lessons, and constraints.
project_decisions	Formal project decisions with status, owner, rationale, and effective date.
project_corrections	Explicit user/engineer corrections to prior AI output or system assumptions.
project_events	Chronological episodic memory and timeline events.
project_summaries	Compact summaries by system/domain and current open issues.

Table	Purpose
documents	Document identity, revision, discipline, status, dates, and ownership metadata.
document_chunks	Chunk text, source location, metadata filters, and vector embedding reference.
memory_evidence	Links memory records to exact source documents, pages, comments, records, or user actions.
memory_audit_log	Immutable history of create, edit, confirm, reject, supersede, and role-based actions.

Suggested project_memories schema

```

id
project_id
memory_type
system
category
content
status      # candidate | confirmed | superseded | rejected
importance  # 1..10
confidence  # 0..1
source_type
source_id
source_page
supersedes_memory_id
valid_from
valid_to
is_current
is_pinned
created_by
created_at
updated_at

```

7. Evidence and Trust Model

Memory in an engineering platform must be explainable. The platform should distinguish between verified evidence and AI interpretation.

Priority	Source class	Typical authority
1	Authority-approved or consultant-approved document	Highest
2	User-confirmed project decision / lead engineer instruction	Very high
3	Latest consultant correspondence or official reply	High

Priority	Source class	Typical authority
4	Latest project drawing, specification, calculation, or submittal	High
5	Historical confirmed project memory	Medium
6	Global company knowledge or historical precedent	Supporting
7	AI inference	Lowest - never authoritative by itself

Critical rule: An AI-generated statement should not become a confirmed engineering memory unless it is supported by evidence or explicitly confirmed by an authorized user.

Example evidence-backed memory:

Memory: Exit light model NEXI250-CGS approved
 System: Emergency Lighting
 Status: Confirmed
 Source: Material Submittal EML-MS-R1
 Page: Consultant approval page
 Evidence: Consultant stamp / approval status
 Confidence: 1.00

8. Document and RAG Integration

The RAG layer should complement the structured memory, not replace it. Documents are ingested, classified, chunked, enriched with metadata, embedded, and indexed for retrieval.

```

Document
|
Classification
|
Parsing / Extraction
|
Chunking
|
Metadata enrichment
|
Embedding
|
Vector index
|
Filtered semantic retrieval

```

Required chunk metadata	Why it matters
project_id	Prevents cross-project contamination.
system	Allows fire alarm, EML, CCTV, FF, etc. filtering.
document_type	Differentiates specification, submittal, drawing, letter, comment sheet, calculation, etc.
document_id / revision	Supports revision awareness and historical queries.
floor / zone	Enables floor-specific drawing and interface retrieval.
manufacturer / model	Improves product and material retrieval.
consultant_status	Prioritizes approved/current evidence.
page / location	Enables direct traceability to source evidence.
date / latest flag	Supports freshness-aware ranking.

Retrieval pattern: Use metadata filtering first (project, system, revision, status), then semantic ranking. Do not rely on embedding similarity alone.

9. Episodic Memory and Project Timeline

Episodic memory records what happened over time and is essential for questions such as "What changed since R1?", "Why was this changed?", and "What did the consultant reject before?"

Event type	Example
design_change	Six detectors added to Level 6 and Level 12.
approval	Material Submittal R1 approved.
consultant_comment	Outdoor duct detector requires NEMA 4X confirmation.
revision_change	R2 shop drawing uploaded and marked latest.
user_correction	Level 7 continues the previous loop; do not open a new loop.
calculation_change	Battery calculation revised after exit sign model change.
interface_change	FA interface schedule revised for a floor or equipment item.

Suggested project_events schema

```

id
project_id
event_type
system
description
related_entity_type
related_entity_id
source_id
created_by
timestamp

```

10. Project Summary Memory

The project summary is a controlled compression layer. It reduces context size while preserving the current operational state.

PROJECT SUMMARY

General

- Project type, consultant, contractor, authority

Fire Alarm

- Current system / brand
- Critical design rules
- Current loop strategy
- Recent changes

Emergency Lighting / CBS

- Current system / brand
- Current calculation assumptions

Approvals

- Latest approved MS / SD / key decisions

Open Issues

- Unresolved consultant or technical items

Recent Changes

- Only significant updates since last summary refresh

Summary refresh triggers should include significant events only: new approval, revision update, user correction, critical design change, new unresolved issue, resolved issue, or changed project fact.

11. Retrieval and Context Building

The ProjectContextBuilder is the central service that decides what information is required for each AI request.

11. Retrieval and Context Building

The **ProjectContextBuilder** is the central service that decides what information is required for each platform action. Because the platform is button/workflow driven, every AI request starts with an explicit Active Task Context.

```

USER ACTION / BUTTON CLICK
|
Create ACTIVE TASK CONTEXT
(project_id, task_type, system, revision, floor/zone,
selected entity IDs, attachment IDs, user inputs)
|
Retrieve structured facts
|
Retrieve relevant memories / corrections / recent events
|
Add project summary when useful
|
Retrieve Project RAG only if documents are needed
|
Retrieve Global Knowledge only if required
|
Apply authority + revision + freshness ranking and token budget
|
Build final context > Agent / LLM

```

Active Task Context should be short-lived. Store only what is needed to continue the workflow: task ID, project ID, task type, current step, revision, floor/zone, selected files/entities, user choices, and temporary outputs. Promote only meaningful results into persistent Project Memory.

Example platform action	Minimum retrieval path
AI Compliance - check detector	Task context + project facts + latest specification + datasheet + approved brand/corrections + global OEM knowledge if needed.
Drawing Review - Level 6 R2	Task context + latest R2 drawing + floor mapping + prior comments + current design rules + unresolved issues.
Complete loop addressing	Task context + project facts + loop strategy + corrections + recent design events.
Battery calculation	Task context + approved equipment + connected loads + circuit distribution + calculation rules + recent design changes.

Indicative token budget

Context component	Typical budget
Active Task Context	100-400 tokens
Project summary	600-1,000 tokens
Structured facts	300-600 tokens
Relevant memories / corrections / events	900-1,800 tokens
Project RAG	2,000-4,000 tokens
Global RAG	1,000-2,500 tokens
Task instructions	300-800 tokens

12. Conflict, Revision, and Correction Handling

The system must preserve historical truth while exposing one current state by default.

```

Old memory
FA Brand = Eaton
status = superseded
is_current = false

New memory
FA Brand = Edwards
status = confirmed
is_current = true
supersedes_memory_id = <old_id>

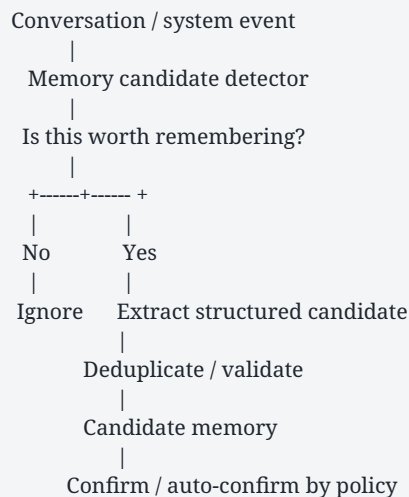
```

- **Revision rule.** Default retrieval uses the latest current revision unless the user explicitly asks for an earlier revision.
- **Conflict rule.** A higher-authority, newer, project-specific source may supersede a lower-authority or older memory.
- **Correction rule.** A user-confirmed correction is stored explicitly and receives high retrieval priority within that project.
- **History rule.** Superseded information is retained for audit and historical questions; it is not silently deleted.

Example: If an AI answer says "max devices per loop = 125" and the user corrects it to 105, the correction is stored as a project memory with a high trust score and should override future inference for that project.

13. Automatic Memory Extraction

Automatic extraction should be introduced only after manual memory workflows, evidence handling, and retrieval are stable.



Candidate extraction output example

```
{
  "remember": true,
  "type": "design_change",
  "system": "fire_alarm",
  "content": "Six detectors added at Levels 6 and 12.",
  "confidence": 0.97,
  "requires_confirmation": false
}
```

Do not store conversational filler such as "OK", "continue", acknowledgements, duplicate instructions, or transient model reasoning.

14. Memory Ranking and Deduplication

Relevant retrieval should consider more than semantic similarity.

```
Retrieval Score =
  semantic_relevance
  x authority_weight
  x importance_weight
  x freshness_weight
  x revision_validity
  x project_scope_match
```

Signal	Purpose
Semantic relevance	How closely the memory matches the current task.
Authority	Approved document / user-confirmed decision should outrank AI inference.
Importance	Critical design constraints outrank minor notes.
Freshness	Recent valid project state usually outranks old state.
Revision validity	Latest current revision is preferred by default.
Project scope	Must be exact for project memory; no fallback to another project.

Deduplication should compare a new candidate with existing memories of the same project, system, and type. Semantically equivalent memories are merged or updated instead of creating repeated records.

15. User Experience and UI

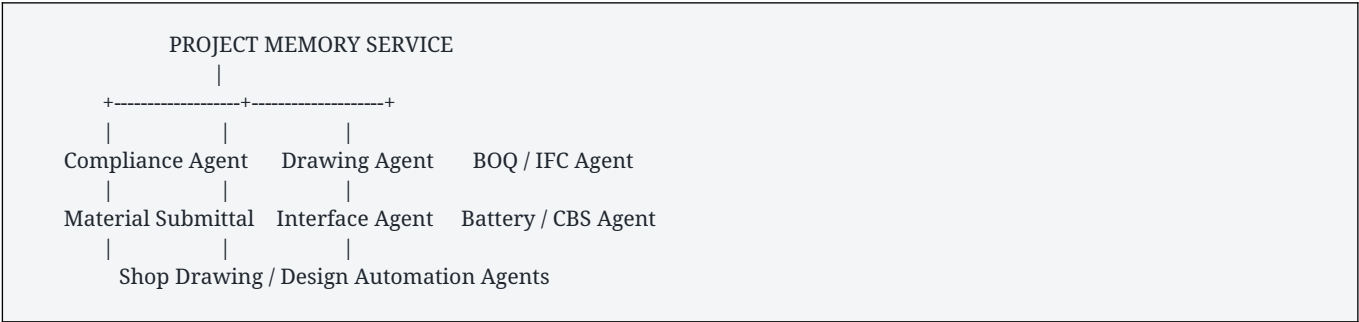
Project Memory should be a visible platform capability while also operating silently behind every project tab.

PROJECT MEMORY	
Overview	
Facts	
Decisions	
Approvals	
Corrections	
Design Changes	
Open Issues	
Lessons Learned	
Timeline	
Filters: System Type Status Revision Date	
Actions: Add Edit Confirm Reject Pin View Source	

UI component	Function
Memory dashboard	Shows counts for facts, decisions, approvals, corrections, unresolved issues, and recent changes.
Memory cards	Display content, system, status, importance, source, timestamp, and revision.
Source viewer	Opens the exact source evidence when available.
Candidate review queue	Allows engineers to confirm/reject AI-proposed memories.
Timeline	Chronological project events with before/after context.
Open issues board	Tracks unresolved technical or consultant items until resolution.
Memory search	Searches within the project memory layer without scanning all raw files.

16. Agent Integration

All AI agents should consume the same Project Memory Service. Agents should not implement independent memory stores or inconsistent retrieval rules.



Agent	Key memory and task-context needs
Compliance Agent	Task type, project scope, VE status, approved brands, consultant history, specs, datasheets, corrections.
Drawing Review Agent	Selected floor/revision, latest drawing, prior comments, design rules, interfaces, unresolved issues.
Shop Drawing Agent	Design constraints, loop/circuit strategy, floor history, user corrections, approved equipment, latest IFC/shop data.
BOQ / IFC Agent	Selected IFC/source, current equipment state, extraction status, system mapping, exclusions, latest quantity decisions.
Battery / CBS Agent	Selected panel, connected loads, circuit distribution, calculation rules, approved equipment, design changes.
Interface Agent	Selected floor/equipment, discipline source, missing drawings, consultant decisions, revision status.

17. API and Service Design

The backend exposes both persistent memory APIs and short-lived task/workflow APIs. Task context must reference the current project and must never become project memory automatically.

```

GET /projects/{id}/memory
POST /projects/{id}/memory/retrieve
POST /projects/{id}/memory/rebuild-summary

POST /projects/{id}/tasks/context
PATCH /tasks/{task_id}/context
POST /tasks/{task_id}/execute
POST /tasks/{task_id}/complete

```

Task-aware retrieval request example

```

POST /projects/15/memory/retrieve
{
  "task_type": "drawing_review",
  "system": "fire_alarm",
  "revision": "R2",
  "floor": "L6",
  "selected_file_ids": ["sd-r2-l6"],
  "include_documents": true
}

```

Retrieval response shape

```
{
  "summary": {...},
  "facts": [...],
  "memories": [...],
  "corrections": [...],
  "events": [...],
  "rag_chunks": [...],
  "evidence": [...]
}
```

18. Security, Roles, and Auditability

Memory is operational project data and requires the same access controls as the rest of the platform.

Role	Recommended permissions
Admin	Manage global memory, edit system policy, configure trust rules, manage all project memories.
Lead Engineer	Confirm critical project facts, approvals, engineering constraints, and corrections.
Engineer	Add/edit project memories, propose facts, resolve issues within assigned projects.
Viewer	Read project memory and evidence only.
AI system	Propose candidate memories; auto-confirm only low-risk types under explicit policy.

- Use row-level project authorization to prevent unauthorized project access.
- Log every memory mutation in `memory_audit_log` with user, timestamp, old value, new value, and reason when applicable.
- Use soft-delete / inactive / superseded states instead of destructive deletion for engineering history.
- Separate permission to modify global company knowledge from permission to modify project-specific memory.

19. Quality, Testing, and Evaluation

Memory quality should be tested as an engineering system, not only as an AI feature.

Test	Acceptance behavior
Project isolation	Project B never retrieves Project A-specific memory.
Revision correctness	Current queries use latest valid revision; historical queries retrieve the requested older revision.

Test	Acceptance behavior
Correction priority	User-confirmed corrections override prior AI inference.
Evidence traceability	Critical memory points to its source evidence.
Conflict handling	Superseded and current facts are clearly separated.
RAG filtering	Search is filtered by project/system/revision/status before semantic ranking.
Task-context isolation	A task executes only against its explicit project and selected workflow state.
Ephemeral-state rule	Temporary workflow selections do not become persistent project memory unless promoted.
Token budget	Context stays task-proportional without losing required evidence.
Agent consistency	Multiple agents retrieve the same confirmed project fact.

Critical test scenarios

Project A: Fire Alarm = Edwards
 Project B: Fire Alarm = Siemens
 Task opened in B -> MUST retrieve Siemens only.

Drawing Review: active floor = L6, revision = R2
 Context Builder -> MUST NOT silently pull L7 or R1 unless required as history.

Temporary selection: user tests a different detector model
 Task context ends -> model is NOT a confirmed project fact unless approved/promoted.

AI assumption: Max devices = 125
 User correction: Max devices = 105
 Future project queries -> 105

20. Implementation Roadmap M1-M14

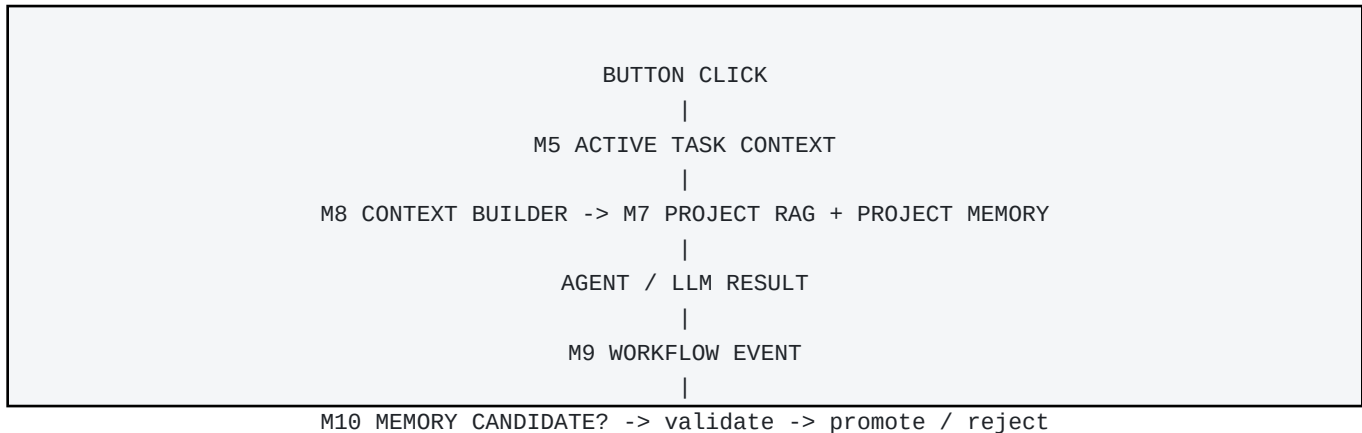
The roadmap now separates persistent project memory from short-lived workflow state. It builds deterministic foundations first, then task context, retrieval intelligence, automation, integration, and production governance.

Milestone	Name	Primary scope	Exit criteria
M1	Project Memory Database	Core tables, enums, project_id scoping, migrations, repository/service layer.	CRUD works; no cross-project retrieval.
M2	Structured Project Facts	Consultant, project type, brands, rules, current revisions, scope, VE, authority.	Facts readable by API without LLM.
M3	Manual Memory UI	Project Memory tab; add/edit/confirm/reject/pin, filters and cards.	Engineers can manage memory manually.
M4	Evidence & Trust	Link memories to documents/pages/records/actions; trust levels and source viewer.	Critical memory is evidence-traceable.

Milestone	Name	Primary scope	Exit criteria
M5	Active Task Context & Workflow State	Create task_run/workflow state model: project, task type, system, revision, floor/zone, selected entities/files, user choices, temporary outputs, expiry.	Every button-driven AI action has explicit, isolated temporary context.
M6	Project Summary Memory	Generate compact summaries by system, approvals, changes and open issues with refresh triggers.	Compact summary available to context builder.
M7	Project RAG Integration	Chunking + metadata pipeline; vector retrieval filtered by project/system/revision/status.	Document retrieval is project-safe and revision-aware.
M8	Task-Aware Retrieval Engine	Implement ProjectContextBuilder, authority/freshness ranking, token budgets and task-specific retrieval routes.	Each workflow receives minimum relevant context.
M9	Episodic Memory & Workflow Timeline	Project events, change history, revision hooks and task-result events.	What/when/why history is queryable.
M10	Automatic Memory Candidate Extraction	Detect candidates from workflow results, approvals, corrections and meaningful system events; deduplicate and review.	Useful results become candidates without storing workflow noise.

Key distinction introduced in M5

Workflow State is temporary. It exists so the next action knows what the user selected and what step is active. **Project Memory is persistent.** Only confirmed facts, decisions, approvals, corrections, meaningful events, and validated lessons are promoted into it.



Milestone	Name	Primary scope	Exit criteria
M11	Conflict, Revision & Correction Resolution	Supersession rules, validity periods, authority ranking, user-correction priority, historical queries.	Current vs historical state is reliable.
M12	Agent & Platform Workflow Integration	Connect Compliance, Drawing, BOQ, Interface, Battery, MS and Shop Drawing workflows to shared task context + memory service.	All workflows use one project state and one context contract.
M13	Task Result Feedback & Memory Promotion	Standardize completion events; decide which outputs become facts, events, open issues, corrections or candidate memories.	Completed tasks improve project memory without saving transient state.
M14	Analytics, Quality & Production Governance	Retrieval precision, false-memory review, token use, task-context errors, correction rate, issue aging, audit and production monitoring.	Quality is measurable and continuously improved.

Recommended execution waves

Wave	Milestones	Goal
Foundation	M1-M4	Safe project-scoped storage, manual control, evidence and trust.
Workflow Context	M5-M6	Explicit temporary task state plus compact project summary.
Intelligence	M7-M9	RAG, task-aware context building and project/workflow event history.
Automation	M10-M11	Memory capture plus robust conflict/revision handling.
Integration	M12-M13	Connect all platform workflows and promote useful results into memory.
Scale	M14	Operate with measurable quality, observability and governance.

21. Deployment Strategy

Deploy the memory capability incrementally inside the existing backend. Add Active Task Context as a lightweight workflow-state service rather than a new chatbot/session subsystem.

Backend: FastAPI services for memory CRUD, task context, retrieval, summary generation, workflow execution and agent context building.

Database: PostgreSQL for persistent memory/events/evidence. A task_runs/workflow_state table may use TTL/cleanup for temporary state.

Worker: ingestion, chunking, embeddings, summary refresh, candidate extraction and task-result event processing.

21. Deployment Strategy - continued

Frontend: React/TypeScript sends explicit task context from each button/workflow: current project, action, revision, floor/zone, selected records/files and temporary user choices.

Observability: log task type, context size, retrieval route, selected evidence, token budget, agent outcome, memory promotion decision and mutation history.

22. Success Metrics

Metric	Target / interpretation
Critical false memory rate	Near zero for approvals, authority requirements, consultant decisions and engineering design values.
Wrong-project retrieval rate	Zero. Any occurrence is a severity-1 defect.
Wrong-revision retrieval rate	Near zero; latest-by-default must be deterministic.
Task-context leakage	Zero cross-project leakage; near-zero stale floor/revision/selection reuse.
Transient-to-memory error rate	Near zero; temporary workflow choices must not be promoted without policy/evidence.
Evidence coverage	High for critical memories and engineering decisions.
Retrieval precision	Relevant context appears in top results with minimal irrelevant content.
Context size / token use	Stable and task-proportional rather than growing with project age.
Task completion traceability	Each AI workflow result is traceable to task context and evidence used.
Agent consistency	Same confirmed fact produces consistent behavior across workflows and agents.

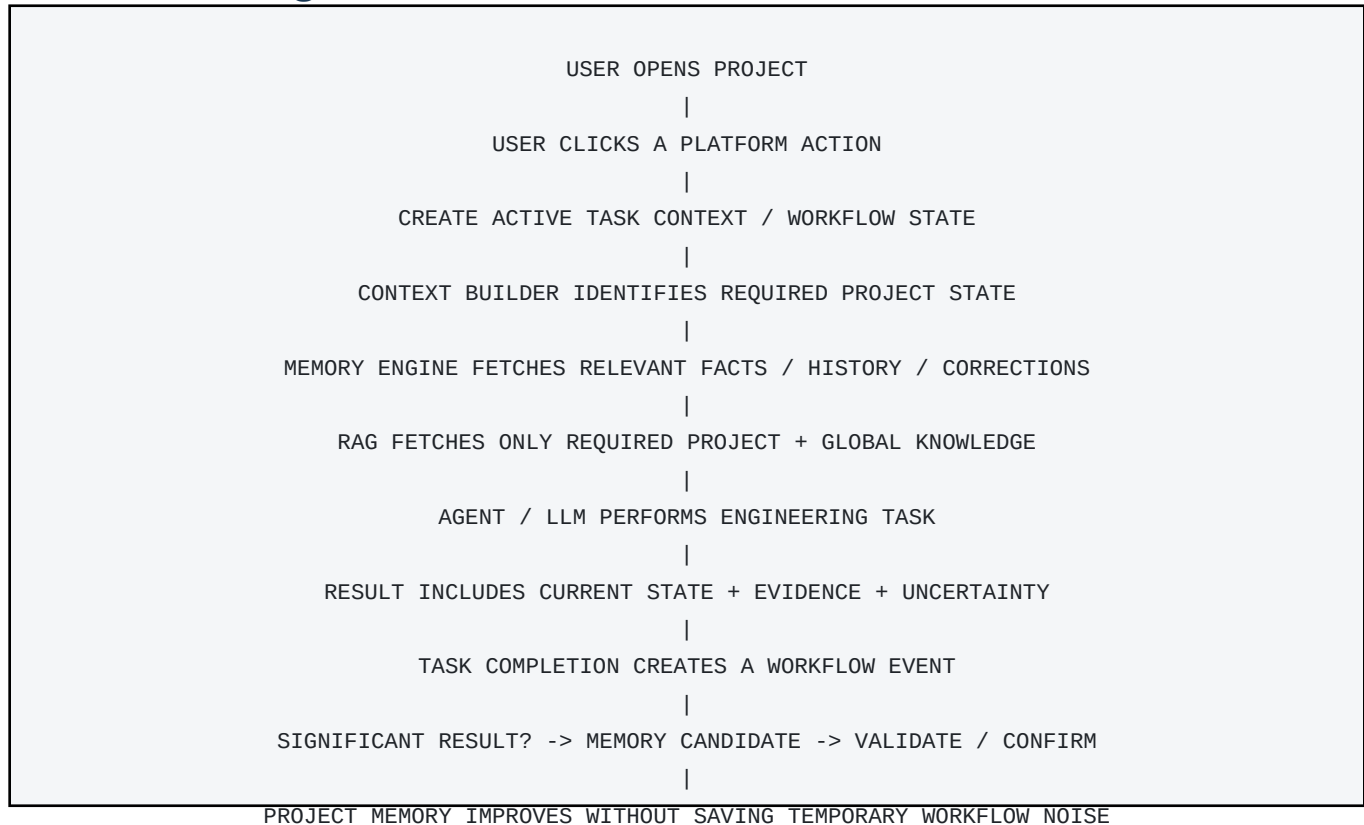
Production acceptance gate

Do not enable broad automatic promotion until M5-M11 tests pass: project isolation, revision correctness, evidence traceability, task-context isolation, correction priority, RAG filtering, and failure behavior when evidence is insufficient.

23. Future Extensions

- Cross-project lessons learned using anonymized, permission-controlled promotion of confirmed patterns.
- Project handover memory packages summarizing final approvals, deviations, unresolved risks, and as-built decisions.
- Change-impact analysis across drawings, calculations, submittals and interfaces.
- Proactive alerts for stale approvals, unresolved comments, revision conflicts or missing evidence.
- Graph relationships between equipment, floors, drawings, comments, submittals, clauses, tasks and decisions.
- Fine-tuned extraction/classification models once a clean supervised dataset exists.

24. Final Target State



Final architectural principle: Project Memory is not chat history and Active Task Context is not long-term memory. Task Context carries the current workflow; Project Memory preserves the trusted engineering state; RAG supplies evidence; the Context Builder joins them only when a task requires it.